

# DBCaaS — Concepts, Architecture & Design Deep Dive

Sep 26, 2026 · @Mohammed Faraaz Shaik

A walkthrough of the real engineering concepts this project has actually used so far, each one grounded in the specific file and code that implements it — not a generic glossary.

## 1. What DBCaaS actually is

The product idea: point it at any tabular data source and get back a data-quality scorecard plus specific flagged rows, without writing source-specific validation code by hand. "As a Service" means the detection logic is centralized and reusable across sources, not that it currently has multi-tenant infrastructure — it doesn't yet, on purpose.

The standing scope freeze is the single most important engineering decision in the project: no database, no auth, no multi-tenancy, no live connectors beyond CSV, no deployment. Everything below was built to prove out the *detection and evaluation* problem first, deliberately isolated from the *productization* problem. This is a real architectural pattern, not just discipline for its own sake: a stateless core has no schema migrations to coordinate, no session state to reason about, and no persistence bugs to chase while the actual hard problem (is the detection any good?) gets solved. Reopening persistence later is additive work; reopening it now would mean touching every layer below simultaneously to add tenant scoping, which is a much bigger and riskier change than it looks from outside.

Concretely, a scan today is one HTTP request in, one JSON response out. Nothing survives between requests. That constraint shaped almost every design decision documented below.

## 2. The connector abstraction (ports and adapters)

`app/connectors/base.py` defines one abstract class, `Connector`, with two methods every source must implement: `test_connection()` and `extract()` -> `list[CanonicalTable]`. `CSVConnector` is the only concrete implementation today, but the interface was written for SQL, REST, and Salesforce connectors that don't exist yet.

This is the ports-and-adapters (hexagonal architecture) pattern: the "port" is the abstract `extract()` contract, the "adapter" is whatever source-specific code satisfies it. The value isn't abstraction for its own sake — it's that everything downstream (canonical models, checks, the agent, the scorer) only ever talks to the port, never to a concrete adapter. The base class's own docstring states the design smell to watch for directly: "if a field like `salesforce_object_type` ever shows up [in the canonical model], the connector abstraction has leaked." That's an unusually explicit, testable definition of what "good abstraction" means in this codebase — not a vague principle, but a specific thing to grep for in review.

Why this matters for a final-year project specifically: it's what makes the claim "schema-agnostic, source-agnostic" true by construction rather than by assertion. A reviewer can open `connectors/base.py`, see the one method every source must implement, and verify the claim in about thirty seconds — that's a much stronger position than a paragraph in a report asserting extensibility.

### 3. The canonical data model

`app/canonical/models.py` defines the "lingua franca" every connector must translate into: `CanonicalTable` (`tenant_id`, `source_id`, `table_name`, `columns`, `rows`) built from `ColumnSchema` (`name`, a closed `ColumnType` enum, `nullable`, an optional `semantic_hint` like "email" or "phone"). This is a classic data-engineering pattern: normalize heterogeneous sources into one shape at the boundary, so everything past that boundary is written once against one schema instead of once per source.

Two details worth understanding, not just noting:

- `nullable` isn't just a type-correctness flag — it's reused later as the field-criticality signal that drives scoring weight (Section 6). One piece of schema information, two different downstream uses. That's a deliberate reuse, not an accident: the alternative was inventing a separate "criticality score" concept, which would be guessing at a new taxonomy instead of using a fact the schema already states.
- `semantic_hint` is how a format-validity check knows a column is an email without hardcoding a column name like `email_address` — the check asks "does any column have `hint=='email'?`" rather than assuming a naming convention. This is what keeps the deterministic checks source-agnostic in the same way the connector interface keeps extraction source-agnostic.

`tenant_id` and `source_id` are present on every table even though there's no multi-tenancy today — provenance is threaded through from the very first connector call so that adding real multi-tenancy later doesn't require touching this model at all, only what's built on top of it.

## 4. The agent pipeline: plan, execute, registry

`GeneratorAgent` ( `app/agent/generator.py` ) splits into two explicit steps: `plan()` decides which checks apply to this table, `execute()` runs them and collects results. This plan/execute split is a real agent-design pattern (distinct from a plain function that just runs everything) because it makes the deciding-what-to-do step inspectable and swappable on its own — the code's own comment says this directly: `plan()` is "intentionally the same orchestration a standalone scan endpoint would have needed," and it's the method that grows when the LLM layer gets added, while execution stays untouched.

Each deterministic check ( `NullCheck` , `DuplicateCheck` , `FormatValidityCheck` , `OutlierCheck` , `ReferentialIntegrityCheck` ) is a self-contained class implementing `applies_to()` and `run()` , registered via a `@register` decorator into a module-level list ( `app/checks/base.py` ). Adding a new check means writing one new class and decorating it — nothing else in the system changes. This is the open-closed principle in its textbook form (open for extension, closed for modification) and it's also a registry pattern / plugin architecture: `GeneratorAgent.plan()` doesn't know the names of any specific check, it just asks every registered one "do you apply here?"

Worth being precise about a limit here, since it matters for how this project should be described: `plan()` today is rule-based ("does this check apply, yes or no"), not an LLM deciding what to investigate. The semantic reasoning layer that runs alongside it is a second, additive capability — always-on when an LLM is configured, not something the planner decides to invoke based on what it finds. That's the accurate distinction between "agent-shaped orchestration" and "autonomous agent," and it's one this project's own documentation is careful about rather than overclaiming.

## 5. The Finding domain model

Before this refactor, the codebase had three unrelated shapes for "something the evaluator found": `CheckResult` (deterministic), `SemanticFlag` (single-table LLM), `CrossTableSemanticFinding` (cross-table LLM). The scorer had to understand all three

individually. `Finding` ( `app/domain/finding.py` ) unifies them behind one representation, built via converter functions ( `findings_from_check_result` , `findings_from_semantic_flags` , `findings_from_cross_table_semantic` ), so the scorer scores off one list regardless of which layer produced each entry.

Two design decisions here are worth understanding as general software-design technique, not just project trivia:

- **Additive refactor, not a replacement.** The old types ( `CheckResult` , `SemanticFlag` , etc.) keep every existing field untouched; `Finding` is built *from* them, alongside them, not instead of them. This is what let the refactor happen without changing the public API response shape or the frontend at all — a textbook example of the Strangler Fig pattern (introduce the new thing beside the old, migrate consumers gradually, never a big-bang rewrite).
- **Deterministic identity via content hashing.** `generate_finding_id()` builds a SHA-256 hash from ( `check_type` , `category` , `affected_dataset` , `rows` , `fields` ) — sorted first, so the same finding always hashes the same way regardless of set/list ordering upstream. This is why a before/after regression comparison is *provable*: the same dataset run through the same code path twice produces identical Finding IDs, rather than relying on a random UUID that would make "did behavior change" an assertion instead of a measurement. Content-addressable identity is the same underlying idea git uses for commits, or that content-delivery caches use for deduplication — identity derived from content, not assigned arbitrarily.

`severity` on a `Finding` is explicitly documented as informational/display-only, derived separately from the scorer's own numeric weight (Section 6) rather than the weight being reverse-engineered from severity. That separation is deliberate: it means a future change to how severity is *labeled* for humans can never silently change a *score*, because scoring never reads that field at all.

## 6. Scoring methodology

`app/scoring/scorer.py` computes, per metric (completeness, uniqueness, validity, consistency, accuracy):

```
score = 100 * (1 - flagged_weight / total_rows)
```

The interesting engineering is in how `flagged_weight` is built, not the formula itself:

| Signal                                                | Weight                      | Why                                                                                    |
|-------------------------------------------------------|-----------------------------|----------------------------------------------------------------------------------------|
| Deterministic flag, no field attributed               | 1.0                         | Flat weight — e.g. DuplicateCheck, where duplication is a whole-row property           |
| Deterministic flag on a required (non-nullable) field | 2.0                         | Reuses the schema's own <code>nullable</code> fact (Section 3) as a criticality signal |
| Deterministic flag on an optional field               | 1.0                         | Same signal, lower stakes                                                              |
| Semantic flag, verifier CONFIRMED                     | 1.0                         | A second, independent model agreed                                                     |
| Semantic flag, verifier NEEDS_REVIEW                  | 0.5                         | Genuine uncertainty is real information, not zero                                      |
| Semantic flag, no verifier ran                        | 0.5                         | An unconfirmed claim from one model alone shouldn't weigh as heavily as a checked fact |
| Semantic flag, verifier REJECTED                      | 0 (never becomes a Finding) | An independent model actively disagreed                                                |

Two mechanics worth understanding as general data-science practice:

- **Row-level `max()` , not summation.** If a row is flagged twice under the same metric (say, two different checks both touch validity), its contribution is the strongest single signal, not the sum of both. This avoids double-penalizing one row for multiple symptoms of what might be the same underlying issue — a modeling choice, not an oversight. A row flagged under *different* metrics still counts against each independently, since those are genuinely separate quality dimensions.
- **Confidence-weighting turns a binary classifier into a continuous scorer.** Without verification weights, this would just be "count of flagged rows" — a naive accuracy-style metric. Weighting by verification outcome is closer to how a precision-aware system should behave: a claim nobody has checked shouldn't cost as much as a claim two independent models agree on.

The docstring in `scorer.py` states explicitly that this is "deliberately behavior-preserving, not a scoring redesign" — the Finding refactor (Section 5) changed the internal data structure the scorer works with, but the actual weights and formula are untouched, and Section 9 covers how that claim is now automatically verified rather than just asserted.

## 7. The semantic reasoning layer

`app/agent/semantic_reasoning.py` is where an LLM (Groq's `openai/gpt-oss-20b`) judges rows that no rule could catch — "a mismatched category, an implausible combination of values" per the system prompt, deliberately distinct from what deterministic checks already cover (nulls, format errors). Several concepts here are genuinely important, not just implementation detail:

- **Sampling, not full-table LLM reasoning.** Deterministic checks scan every row; the LLM judges a bounded sample per batch (`_DEFAULT_SAMPLE_SIZE = 20`, though production uses a tuned 45 — see below), then the generator loops batches until every row has been sampled at least once. This is a real cost/latency engineering tradeoff: LLM calls are the expensive, slow part of the pipeline, so the design spends them deliberately rather than on every row for every check.
- **The batch size itself was tuned empirically, not guessed.** The code comment documents a real experiment: batch size 20 gave recall 0.756–0.896 across three forged datasets; size 100 hit Groq's token cap and auto-shrank to ~48–50 rows in practice, with recall 0.854–1.0 at that landing size — likely because larger batches let the model cross-reference rows against each other, which the prompt explicitly asks it to do. 45 was picked as a conservative number near that empirically-good zone. This is a small but real example of hyperparameter tuning against a held-out evaluation set, the same underlying discipline as tuning a model's batch size or learning rate against a validation metric, just applied to a prompt-based system instead of a trained one.
- **Prompt-injection defense, built into the prompt itself, not bolted on after.** Row data is explicitly framed as `UNTRUSTED DATA` in the prompt, with an instruction that a claim embedded in a field ("pre-approved," a fake "SYSTEM MESSAGE," "ignore previous instructions") can never change the flagging decision — and text that *looks* like an injection attempt is itself treated as a high-confidence signal to flag, not something to fall for. This mirrors real prompt-injection mitigation practice: never trust content that flows through an LLM's context window as if it were a system-level instruction, no matter how it's phrased.

- **Forced structured output, fail-safe parsing.** The prompt demands JSON-only output ( `{"flags": [...]}` ), which is what lets a verifier later check each flag against actual evidence rather than trusting unstructured prose. If the model returns something malformed, `_parse_response` fails to zero flags rather than crashing the scan — a deliberate fail-safe default (never let a flaky model response take down an otherwise-working deterministic scan).

## 8. Verification and confidence derivation

**LLM-as-judge, with a different model.** Every semantic flag from the generator (Groq) is independently re-checked by a verifier (Gemini) before it's trusted — a different model checking the same claim, not the same model grading its own work. This is the LLM-as-judge pattern, and using a *different* provider for it specifically avoids the failure mode where a model is systematically overconfident about its own reasoning in a way a second call from the identical model wouldn't catch.

**Why raw LLM confidence isn't trusted as a probability.** `app/domain/confidence.py` documents a concrete, observed failure: four structurally different flagged rows (one obvious mismatch, three subtle ones) all came back from the generator with `confidence=0.95`. A model reporting an identical number regardless of case difficulty isn't calibrated — it's a canned value. Treating that as "95% likely to be real" would be false precision dressed up as a number.

**The fix: heuristic derivation from structural facts, not the model's own claim.**

`derive_confidence()` starts from a base rate per check type (deterministic=1.0, single-table semantic=0.60, cross-table semantic=0.50, since cross-table reasoning requires resolving a foreign key *and* judging cross-table consistency — more places to fail), then adjusts based on what actually happened: a genuine CONFIRMED from the verifier raises it; a genuine NEEDS\_REVIEW (verifier ran, was honestly uncertain) leaves it unchanged, because uncertainty is real information, not an absence of it; a NEEDS\_REVIEW caused by the verifier *technically failing* (an API error, a quota exhaustion) lowers it below even an unverified flag, because that case has *less* evidence than an ordinary unverified claim, not equal evidence with a different label. This distinction only exists because a real production incident (Gemini free-tier quota exhaustion mid-scan) revealed both cases were previously indistinguishable downstream.

**The important naming discipline here:** this module and its output field were originally called `calibration.py` / `calibrated_confidence`, and were deliberately renamed to `confidence.py` / `derived_confidence` after review. "Confidence calibration" is an

established statistical term — measuring predicted confidence against empirical correctness (a reliability diagram, Expected Calibration Error) and correcting the gap. This module doesn't do that; there's no labeled outcome dataset to calibrate against yet. Naming it "calibration" would have been claiming rigor that doesn't exist. This is a genuinely important lesson for any technical project: a name is a claim, and a name that overclaims what the code actually does is a correctness bug in the documentation layer, not just a style nitpick.

## 9. Evaluation methodology

This is the part of the project closest to applied data science, and it's built in layers of increasing sophistication:

| Eval                                      | What it tests                                                                                                                               | Key idea                                                              |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <code>run_eval.py</code> (v1 adversarial) | Basic recall against forged ground-truth issues                                                                                             | Established the baseline: can the system find planted problems at all |
| <code>run_schema_eval.py</code>           | Structural correctness (cross-table findings match planted orphaned FKs exactly; aggregate score really is the row-weighted mean it claims) | Needs no LLM — fast, deterministic, runnable in CI                    |
| <code>run_adversarial_v2_eval.py</code>   | Resistance to evasion, not just detection                                                                                                   | See below                                                             |

**Adversarial v2's real contribution: a paired differential design.** Rows come in two variants — a plain "attack" version (the issue plus an evasion technique: semantic paraphrase, lexical obfuscation, logical reframing, contextual camouflage) and a "control" version (the same issue, no evasion). A row flagged in attack-but-not-control means the evasion technique actively suppressed detection — a real FAIL. A row flagged in *both* is logged as a general false positive, but deliberately does *not* fail the eval, because this benchmark measures injection/suppression resistance specifically, not general precision — that's a different, legitimate benchmark for a different day. This paired-control design is

the same underlying idea as an A/B test or a case-control study: isolate one variable (the evasion technique) by holding everything else constant, rather than comparing unrelated positive and negative examples.

**Ground truth as ordinary JSON, not a black box.** `ground_truth_v2.json` states, per row, `has_real_issue`, `must_not_flag`, `role`, and `injection_technique` explicitly — so the eval script's pass/fail logic is auditable by reading two files side by side, not a hidden scoring function.

**Regression tripwires — turning a one-off manual check into a permanent, automated one.** The Finding refactor (Section 5) was originally verified by manually re-running `run_schema_eval.py` before and after and confirming an identical score (96.0 → 96.0). That was real evidence, but it only existed as something someone had to remember to re-run. It's since been converted into

`test_full_pipeline_frozen_baseline_catches_scoring_drift` — a pytest test that runs the real pipeline against a small fixed dataset and asserts exact pinned scorecard values, so any future change that silently alters scoring behavior fails the test suite automatically. This is a specific, transferable lesson: a claim proven once by hand is weaker engineering than the same claim enforced by a test that runs every time, and converting the former into the latter is real, valuable work even when the underlying fact hasn't changed.

**Versioning as the reproducibility mechanism.** `app/version.py` hand-bumps constants (`DETECTION_VERSION`, `SCORING_VERSION`, `SEMANTIC_PROMPT_VERSION`, `CROSS_TABLE_PROMPT_VERSION`, `CONFIDENCE_DERIVATION_VERSION`) attached to every scan response. Without a database, this is what lets someone looking at a scorecard later answer "which exact logic produced this number" — reproducibility solved through metadata that travels with the response, rather than through persistence.

## 10. Engineering discipline and scope

**Statelessness is architecture, not a missing feature.** Every domain object in this codebase — `CanonicalTable`, `ScanResult`, `Finding`, `ScoreCard` — has no database ID, no session reference, nothing that assumes persistence. A scan is a pure request-in/response-out computation. This is what made the Finding refactor, the confidence-derivation rename, and the versioning system all safe to build quickly: none of them had to reason about existing rows in a database that might already reflect the old behavior. The

tradeoff is explicit and named throughout the code: reopening persistence later touches every layer at once (to add tenant scoping), which is real, acknowledged future cost — not a cost being hidden.

**Test-driven confidence, not test-as-afterthought.** 204 tests currently, and the pattern worth noticing is which claims get a dedicated test versus which don't:

`test_criticality_weighting_end_to_end_with_real_null_check` runs the *actual* `NullCheck` through the *actual* scorer (not mocks) specifically to prove the two layers genuinely connect, not just that each behaves correctly in isolation.

`test_multi_file_scan_request_leaves_no_temp_files_behind` doesn't just check the upload endpoint returns 200 — it wraps `tempfile.NamedTemporaryFile` itself and asserts zero files survive the request, so a *reintroduced* leak would be caught, not just the originally-reported one. That's a meaningfully higher bar than "the bug is fixed": it's "the bug cannot silently come back."

**What's deliberately out of scope, and why that's valid engineering, not a shortcut:**

persistence/database, auth/multi-tenancy, additional live connectors beyond CSV, deployment infrastructure. None of these are missing because they were forgotten — each one is a fundamentally different engineering problem (state management and migrations; identity and access control; per-source integration work; ops and infra) than "is the detection logic itself sound," which is what this phase was built to prove. A real engineering skill, not just a project-management one, is recognizing when a problem should be deliberately fenced off so the problem actually being solved doesn't get diluted — and being able to articulate precisely *why* the fence is where it is, which every module docstring in this codebase does.

**The throughline across all ten sections above:** almost every non-trivial decision in this codebase is documented at the point it was made, with the reasoning, not just the choice — the sampling batch size with its actual sweep numbers, the confidence rename with the exact review comment that triggered it, the `max()`-not-`sum()` scoring choice with why. That habit — writing down *why*, not just *what* — is arguably the single most transferable practice to take out of this project into any future engineering work, independent of any of the specific technologies used here.